

Lab 5 - C4 Design Challenge

Student Sheet

Duration: 90 minutes

AI1220 — Software, Web & Mobile Engineering: Code without Limits

Design brief

The university wants a website for student-led workshops in photography, music and game development. Students need to browse workshops and reserve a place. Organisers need to publish workshop details and see who has reserved a place in their workshops.

Design the **Campus Workshop Board** for a pilot of 200 students, maintained by one small development team. The student who creates a workshop is its *organiser* and owner. Produce four C4 views and one dynamic diagram. **No application code or long report is required.**

Customer rules

ID	Required behaviour
----	--------------------

- | | |
|----|---|
| R1 | Only signed-in university students use the board. External Campus Identity verifies a credential and supplies student ID, name and email. |
| R2 | Students create workshops with a unique ID, owner ID, nonempty title, start time and capacity of 1–20. Only the owner may publish a workshop. Publishing does not reserve the owner a seat. |
| R3 | Students see each published workshop's title, time, capacity, remaining seats and their own reservation. Only its organiser sees the attendee names and emails. |
| R4 | Reserve at most one seat per student per published workshop. Never exceed capacity, even when requests overlap. A verified student's repeat request returns their existing reservation unchanged, even if full, with no extra seat or email. Reject new reservations when full and all unverified requests without changing reservations. |
| R5 | Workshops, ownership and reservations survive restart. Refreshing shows the stored state. |
| R6 | New reservations request confirmation through external University Mail . If mail is unavailable, retain the seat and tell the student that the reservation succeeded but email is unavailable. No automatic retries are required. |
-

Keep to these features.

Setup

Work individually in `ai1220/lab05/`. Extract `lab05_files.zip` there. Use any diagramming tool; save editable sources and readable exports. AI assistance is allowed, but check and understand your work.

Drawing rules

Give each view a title and legend. Name its elements, types and responsibilities; label arrows with the interaction and information exchanged. State technologies and communication mechanisms for containers/components. Use consistent names across views.

An *interface* defines how parts exchange information. Each deeper C4 view expands one named parent. The short scope reminders on page 2 are enough to start.

What to hand in

Five diagram exports (PDF, SVG or PNG), their editable sources, and the short `DESIGN.md` template. Let the diagrams answer the questions; add only the notes requested on page 2. Marks follow the five exercise weights; partial completion earns proportional credit.

1. Where does the system end? 15%

Who uses the board, why, and which systems are outside your team's responsibility? Draw **01-context**: the board as one software system, its users and external systems.

2. What needs to run? 20%

Which applications and data stores do you need, and how do they communicate? Draw **02-containers**. A C4 container is an application or data store. Add **one sentence** justifying a structural choice.

3. Which components should exist? 25%

Inside the application that handles reservations, what parts should exist? First list your own components in **DESIGN.md**: **name, job, rule(s) served**. Then draw **03-components**, showing their interfaces and exchanged data. A component groups related code inside one application; it is not a separate deployed service. Show its parent boundary and relevant supporting elements.

4. What happens inside one component? 15%

Expand your reservation-rules component into a small **04-code** sketch of classes/functions and their relationships. Specify one operation's typed inputs and possible results. What invariant does it preserve, and *why* does that guarantee survive overlapping calls? Answer briefly on the diagram or in **DESIGN.md**. No implementation is required.

5. Can your design explain this incident? 25%

Workshop W17, "Build a tiny game", belongs to S10. It is published, has capacity 3, and S21/S22 already hold seats. Ari (S23, `ari@example.invalid`) is signed in but has no reservation. Ari selects Reserve; **University Mail is unavailable**. Storage works and nobody else acts during this trace.

Using your existing components, what messages are exchanged and what does Ari see? Draw **05-dynamic**: one numbered runtime story, with message data/results. A dynamic C4 diagram reuses static elements to show interaction order; it is not a fifth level. Either collaboration or sequence layout is valid. Record **stored state before, stored state after, and Ari's message** in **DESIGN.md**.

Check and submit

Do the names, responsibilities, interfaces and outcomes agree across all five views? Check that exports are readable and match their editable sources. Save sources in **diagrams/source/**, exports in **diagrams/exports/**, and retain **README.md**. Map any multi-page file in **DESIGN.md**.

Commit and push `lab05/` before the session ends. Check the files on GitHub and the existing instructor access. If a tool or submission fails, keep local work and ask for help; no paid diagramming tool is required.

References: [C4 views](#); [code-level scope](#); [dynamic diagrams](#); [notation](#).